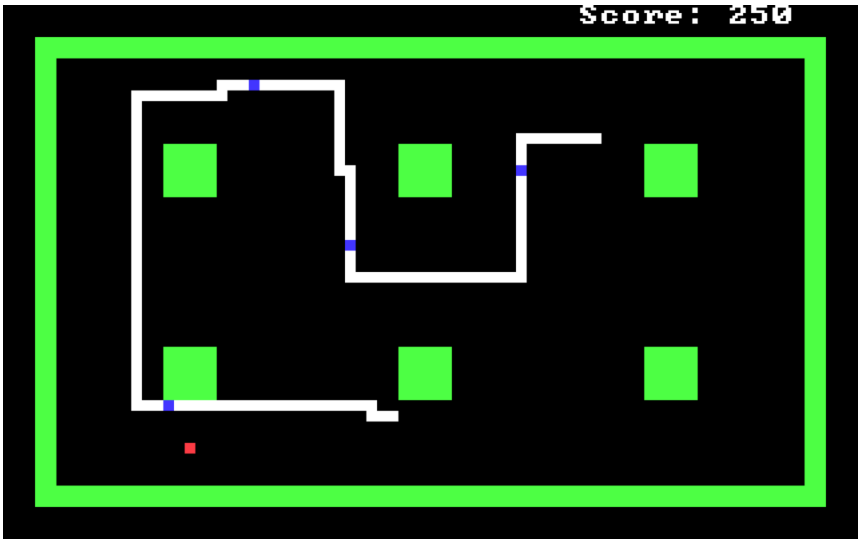
VRIJE
UNIVERSITEIT
BRUSSEL

Snake in assembly



Jeremy Lombaerts

Januari 2017

Wetenschappen & Bio-ingenieurswetenschappen
Departement Natuurkunde

Contents

1 Description **2**

2 Design and development **3**

2.1 Overview 3

2.2 The snake 4

2.3 moveSnake 5

2.4 growSnake 7

3 Problem solving **8**

1 Description

The program presented here was written completely in the x86 Assembly language. It is a clone of the famous Snake game. The user controls a snake, moving over a playing field. The aim of the game is to eat the food on the field and to grow ever bigger. But beware, if the snake crashes into a wall or bites it's own tail, it's game over!

Upon start, the player is faced with a level selector. Currently, there are three levels available. The classic, empty field (1), a checkboard field (2), and a cage field (3). The player can select a level by pushing the appropriate key on his keyboard (1 for the classic field etc.).



Figure 1: Classic

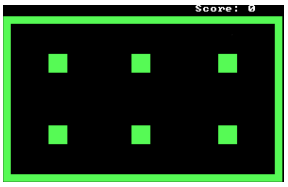


Figure 2: Checkboard

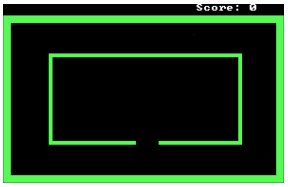


Figure 3: The cage

Once the game begins, the snake is spawned in the middle of the field, always going upwards. The player then controls the snake with the arrow keys. The player can move freely over the field and can eat food. Optionally, the player can quit the game by pressing q or pause the game by pressing p.

When food is eaten, the snake grows (base growth is 5 blocks). Furthermore, if the player is doing well, the game gets harder. Once the player has reached a score of 100, 250 or 500, the snake speeds up!

When the player dies, he is presented with a game over screen, displaying his score and prompting him to either quit the game (q) or start a new game (n).



Figure 4: The game over screen

2 Design and development

2.1 Overview

This section covers the general structure of the code.

The program follows a simple, typical game structure (see fig. 5). The program entry point immediately calls a procedure to welcome the user and present the user the level selection menu.
Once the level choice is confirmed, the program calls a procedure to set up the game, followed by a gameplay procedure.
The gameplay procedure consists of a first part which initialises all variable game elements (the snake's length, coordinates and pointers, and the score) to their begin values. The second part is a standard infinite game loop. The gameloop calls different procedures required to make the game run. Five different procedures are being called. The first procedure checks whether the user has requested a change of direction (with the arrow keys) or pressed p or q to pause or quit the game. Next, the program checks whether there is already food for the snake on the playing field. If there is no food on the field, random coordinates are generated. If these coordinates are not valid (e.g. outside the field, on the snake,...) nothing happens. If the coordinates are valid, food is spawned. This explains why it sometimes takes a little longer for food to reappear on the field. Once the food spawning procedure is done, the program checks whether the snake will not crash into a wall or bite itself. It is important that the program checks for potential collisions AFTER having set the direction in which the snake is going to move, but BEFORE actually moving the snake. This ensures correct behaviour in some situations e.g. when moving against the wall, changing direction just before eating food... The program checks for collisions with the walls and obstacles by comparing the color value of the next square in the video memory with the color set for the walls. Collisions with food or itself are checked by comparing the coordinates of the food, or the snake's head with the x and y coordinates of the snake's squares. The check is performed by looping over the x coordinates and if a match is found, checking if the corresponding y coordinate also matches. This is a little more efficient than always checking both x and y. Finally, the program actually moves the snake and calls a procedure to slow down the game and synchronise it to 60hz. The procedure responsible for moving the snake will be discussed in detail in the next section. The game slowing procedure is used to slow down the game to a reasonable speed. By decreasing the duration of this procedure once a player reaches certain scores, the game gets increasingly harder.



Figure 5: The program structure

2.2 The snake

Conceptually, the snake is a collection of connected squares with given dimensions.

The snake can perform several actions:

- Move: The snake is moved by first deleting the last square of the snake by overwriting it with the background color and then drawing a new square in front of the snake, in the direction specified by the user.
- Grow: The snake grows (by default with 5 squares) when it eats food. this is achieved by making sure that for the next 5 calls of the moveSnake procedure, the last square of the snake is not deleted, whilst the head of the snake still advances.
- Eat: The snake eats it's food by simply overwriting the food color by the snake color in the video memory. To make it look like the snake

swallows it's food, the snake's second square's color is changed to the swallowed food color.

Fundamentally, the snake is stored in memory as two arrays, containing the x and y coordinates of all of the snake's squares, and 3 bytes, containing the snake's length and pointers to the snake's head and tail. At first, this may seem overly complicated, but this approach minimises the amount of read and write operations during the game loop. To understand the exact functionality of this structure, it is needed to investigate the moveSnake and growSnake procedures in detail.

2.3 moveSnake

The snake is moved by first getting the coordinates of the snake's last square by retrieving them from the x and y coordinate arrays. The crucial point of the method used is that the head and tail coordinates are not necessarily the first and last entries of the arrays! To find the snake's tail coordinates, the program provides a 'pointer' to the tail. TSNAKE is in fact the offset of the tail in the array. By retrieving the values stored in [XSNAKE+TSNAKE] and [YSNAKE+TSNAKE], the coordinates of the snake's tail are found and the last square can be deleted.

Once the last square is deleted, that memory slot is available to store the new coordinates of the snake's head when it advances. So the new coordinates are stored at the locations [XSNAKE+TSNAKE] and [YSNAKE+TSNAKE]. The last step is to update the HSNAKE and TSNAKE pointers, They need to decrease by one slot (unless they're pointing to the zero-th slot, in which case they should now point to the last slot).

This method makes sure that every time moveSnake is called upon, the amount of read and write operations is the same. The alternative would be to always have the snake's head as the first entry and the tail as the last. But this would require to shift all the array's entries by one slot each time moveSnake is called. This is inefficient and could slow down the game as the arrays grow larger and larger.

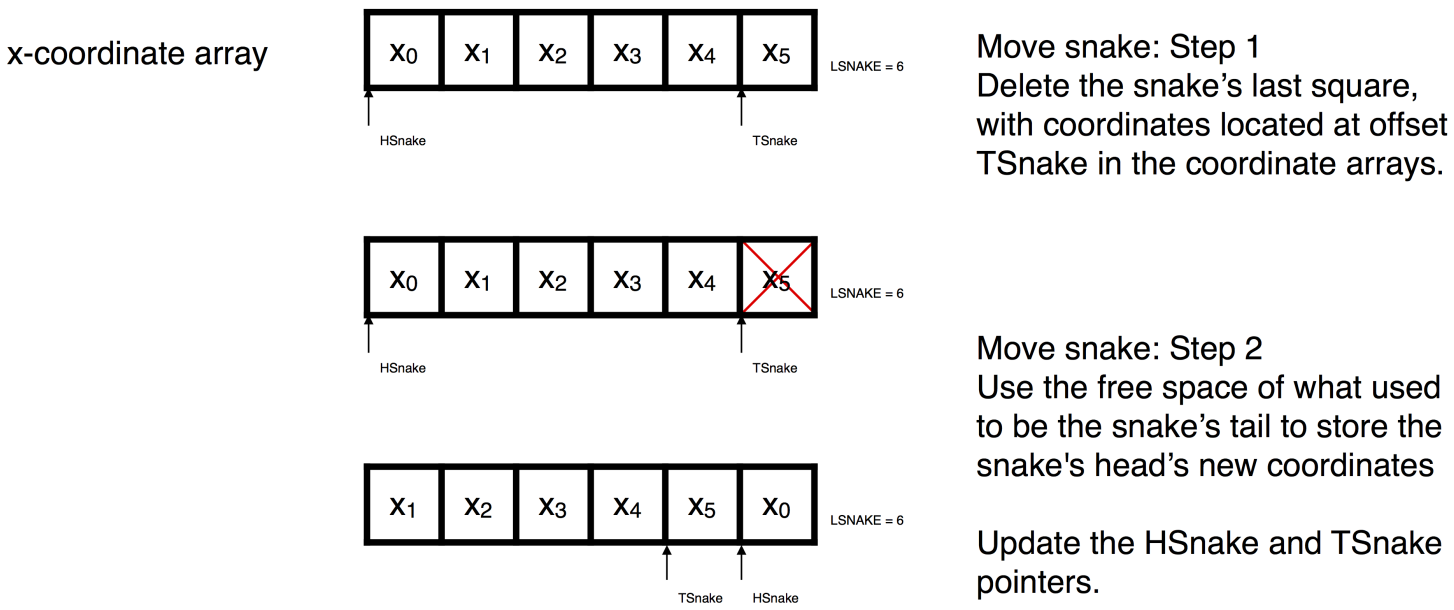
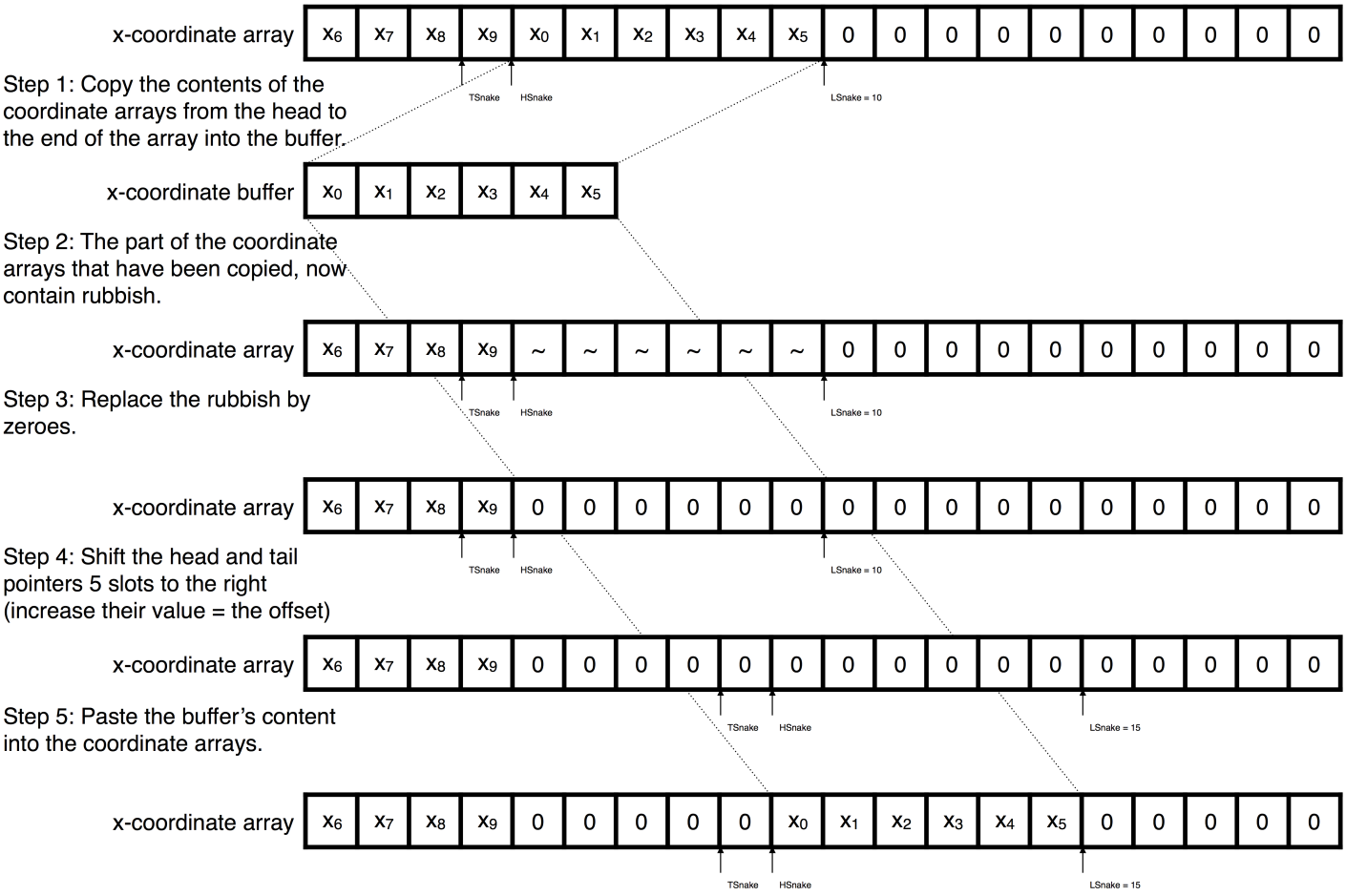


Figure 6: The moveSnake procedure's functioning

2.4 growSnake

The growSnake procedure acts upon the same structure as the moveSnake procedure. It is responsible for making the snake (appear to) grow. The general idea is to insert some empty slots between the snake's tail and it's head. The exact procedure implemented is described in fig. 7.



Step 6: Calling moveSnake will delete the square with coordinates found at TSnake. But these are now zero. Thus the square at (0,0) is deleted, but that square is not a part of the snake so the snake will appear to be growing! This approach reduces as much as possible unnecessary read/write operations.

Figure 7: The growSnake procedure's functioning

3 Problem solving

No major problems were encountered during development. The hardest part was figuring out the data structure used to hold, move and update the snake. Since especially the move function gets called a lot, it needs to be simple, fast and reliable. I believe the data structure presented above meets all these criteria.

Some of the other problems encountered were due to data and register sizes (byte vs. word vs. double word). One fierce bug was caused by using the wrong size when using the TSNAKE and HSNAKE pointers. The XSNAKE and YSNAKE arrays are of the double word kind[1] . If we want a TSNAKE pointer, pointing to for example the 5th entry in the XSNAKE array, TSNAKE should not contain the value '4', but '4 x 4 = 16', because for each slot we want to advance, we need to advance by 4 bytes (= size of a double word) in memory!

Another problem I encountered was the variable execution time of the program. The first version of the slowDownGame procedure was a simple loop:

```
1.      mov ecx, 0
2.      pauseLoop:
3.          nop
4.          dec ecx
5.       jnz pauseLoop
```

But this did not provide a reliable timer. The solution was given to us. The video card refreshes at a rate of 60hz. It is possible to detect when the video card is done refreshing. By waiting until the video card is done refreshing (or let it refresh more than once to slow down even more), it is possible to make a reliable timer.

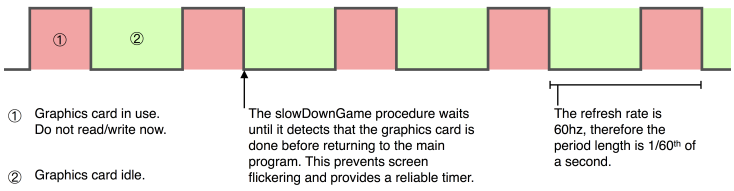


Figure 8: The slowDownGame procedure's functioning

[1]Strictly speaking, in a 320×200 pixels field, the word type would have been enough since the coordinates can not be larger than $2^{16} = 65.536$, but I only figured that out when a large part of the code was already written.

The last major obstacle worth discussing is the generation of food. The generateFood procedure works by first checking whether there is already food on the field. If there is food on the field, a flag is raised. By checking the state of the flag, it is easy and fast to determine whether more food needs to be generated or not.
If the flag is lowered, the procedure continues by generating a random number. This random number lies in the range $[0, 2^{32}]$. Dividing this number with 64.000 and keeping the reminder results in a random number in the range [0, 63.999]. This can be used as an offset in video memory, containing the location of the food to be generated.
The fundamental problem encountered during development was the actual generation of pseudo-random numbers. The first version of the number generator:

```
1.      mov ah, 00h
2.      int 1ah
3.
4.      mov eax, edx
5.      imul eax, eax
```

This generator simply looks at the system time (INT 1AH, function 00h) and takes the value of edx (seconds) and squares if. By squaring it, a pseudo-random number is generated. Obviously, this is not a good generator (although it does the job more or less). Figure 9 compares this crude generator with the implemented solution, a true linear congruential generator.

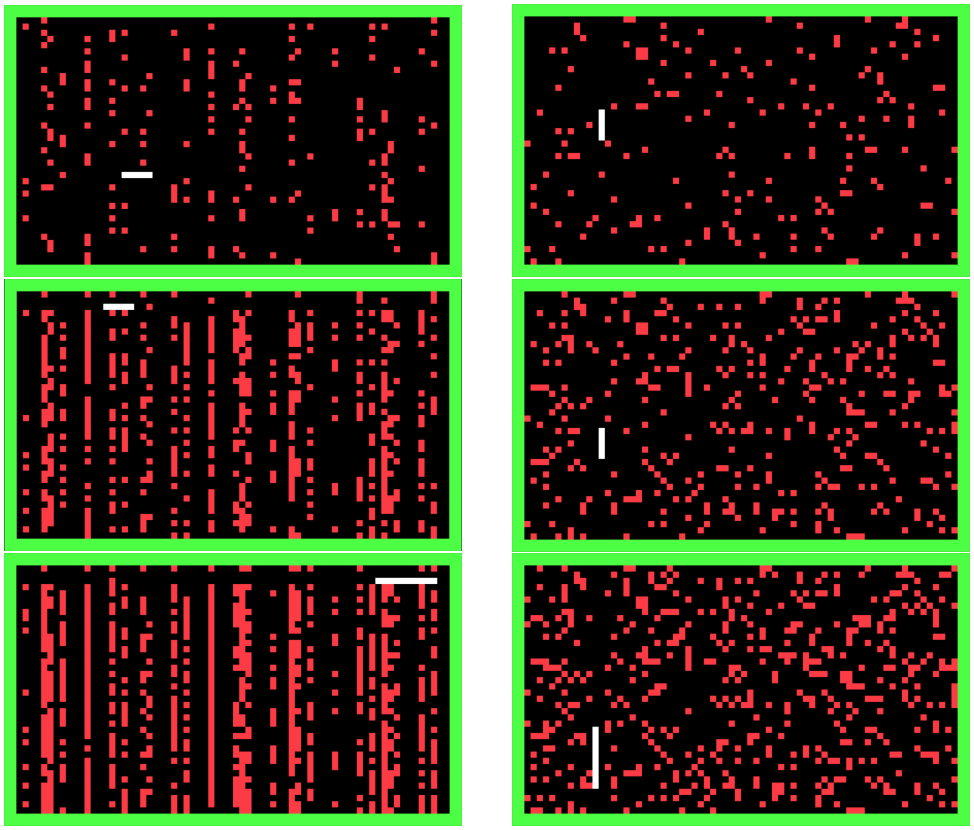


Figure 9: A crude and naïve generator on the left versus a good linear congruential generator on the right. It is clear that the left is not even close to a random distribution.